Failure Modes in Chained Tool-Calling Agents Operating Without Human Checkpoints

A Systematic Review of Reliability, Safety, and Oversight Gaps in Autonomous Multi-Step LLM Agents

Abstract

Large language model (LLM) agents that autonomously plan, invoke external tools, and chain the results of one action into the next have moved rapidly from research demonstrations to production deployments. When such agents operate without human checkpoints—that is, without a person reviewing or approving intermediate steps—small errors in reasoning, tool selection, or parameter generation can compound across a trajectory and produce outcomes that are difficult to detect, attribute, and reverse. This paper synthesizes recent empirical and conceptual literature on failure modes in chained tool-calling agents. We organize failures into four interacting categories: reasoning and planning errors, tool-use and parameter errors, memory and context degradation, and coordination failures in multi-agent settings, and we examine how the absence of human checkpoints changes the severity and detectability of each category. Drawing on taxonomies such as the Multi-Agent System Failure Taxonomy (MAST), the AgentErrorTaxonomy, and Agent-SafetyBench, together with security research on indirect prompt injection and studies of rapidly increasing agent task-completion horizons, we argue that unchecked autonomy amplifies error propagation, converts silent policy violations into consequential actions, and expands the attack surface available to adversarial inputs. We then review current mitigation approaches—deterministic verification gates, reflection and self-critique, structured multi-agent protocols, and tiered human-in-the-loop and human-on-the-loop oversight models—and assess their partial effectiveness. We conclude by identifying open research gaps, including the lack of standardized failure-attribution methodology, the absence of calibrated autonomy-risk mappings for real-world domains, and the need for oversight mechanisms that scale with agent capability rather than being bolted on after deployment.

Keywords: LLM agents; tool calling; autonomous agents; multi-agent systems; error propagation; human-in-the-loop; AI safety; prompt injection; agent evaluation; oversight.

1 Introduction

The past three years have seen a rapid shift in how large language models are deployed: from single-turn text generators to agents that reason iteratively, call external tools and application programming interfaces (APIs), and chain the outputs of one action into the inputs of the next. Foundational frameworks such as ReAct interleave reasoning traces with actions, allowing a model to "think," issue a tool call, observe the result, and continue reasoning in a closed loop [21]. Building on this pattern, systems such as Toolformer [14], Gorilla [9], and ToolLLM [10] demonstrated that LLMs could be trained or prompted to invoke thousands of real-world APIs with reasonable

accuracy. These capabilities have since been packaged into commercial agent frameworks and protocols—including the Model Context Protocol (MCP) and Anthropic's computer-use and coding agents—that allow a single model session to execute dozens or hundreds of chained tool calls without a human reviewing each step [3].

This shift toward longer, unchecked chains of autonomous action is not merely qualitative. Kwa et al. [6] at METR show that the length of software tasks frontier AI agents can complete with 50% reliability has been doubling approximately every seven months since 2019, and that this trend is driven substantially by improved tool use and error-recovery ability rather than raw reasoning alone. As the horizon of unsupervised task execution grows, so does the number of chained tool calls that occur between one point of human awareness and the next. A checkpoint-free agent that plans ten steps ahead, calls five different APIs, and only surfaces a final summary to the user is qualitatively different, from a risk standpoint, than a chatbot that proposes a single action for approval.

This distinction matters because the empirical record on agent reliability is sobering even in controlled benchmark settings. On WebArena, a reproducible benchmark of long-horizon web tasks across four fully functional website categories, the best GPT-4-based agent solved only 14.41% of tasks end-to-end against a human success rate of 78.24% [23]. On the Multi-Agent System Failure Taxonomy (MAST) dataset of over 1,600 annotated traces spanning seven popular multi-agent frameworks, researchers identified fourteen recurring failure modes clustering into specification issues, inter-agent misalignment, and inadequate verification—problems attributable to system design rather than raw model capability [2]. These findings motivate the central question of this paper: what specifically goes wrong when chained tool-calling agents act without a human checkpoint, and what does the literature suggest can be done about it?

The remainder of this paper is organized as follows. Section 2 reviews the technical and governance literature underlying tool-calling agents and human-oversight frameworks. Section 3 develops a structured account of the major failure categories reported in the literature. Section 4 surveys current mitigation approaches. Section 5 discusses research challenges and limitations of the existing evidence base. Section 6 identifies research gaps and future directions, and Section 7 concludes.

2 Background and Related Work

2.1 From Reasoning Chains to Tool-Calling Agents

Chain-of-thought prompting established that decomposing a problem into intermediate reasoning steps improves LLM performance on multi-step tasks (referenced widely in the agent literature as a precursor technique). ReAct extended this idea by interleaving reasoning with external actions, showing that alternating "thought" and "act" steps reduces hallucination relative to reasoning alone and outperforms pure imitation- or reinforcement-learning-based agents on interactive benchmarks such as ALFWorld and WebShop [21]. This reasoning–acting loop is now the dominant architectural pattern underlying commercial agent products: the model proposes a tool call, an executor runs it, the result is appended to context, and the loop continues until the model judges the task complete.

Parallel work focused on making tool invocation itself more reliable. Gorilla fine-tuned a LLaMA-

Table 1: Future Research Directions for Robust LLM Agent Systems

Research Direction	**Core Focus**	**Target Failure Class**	**Key Failure Mode**	**Proposed Strategy**
Dynamic Formal Verification	Runtime invariance	Verification	Silent acceptance of invalid environment outputs	Spec-driven runtime checking & rollback
Cross-Agent Protocol Contracts	Coordination interfaces	Coordination	Cascading errors from unvalidated communication	Typed message contracts with schema verification

based model with retrieval-augmented API documentation and found that it surpassed GPT-4 on the accuracy of generated API calls while reducing hallucinated arguments [9]. ToolLLM scaled this idea to more than 16,000 real-world APIs and introduced a depth-first search-based decision tree to help agents explore multiple reasoning paths and evaluate them before committing to an action [10]. Toolformer showed that models can learn, largely self-supervised, when and how to call auxiliary tools such as calculators, search engines, and calendars [14]. Together, these works established that tool calling is learnable and can be made increasingly precise, but they also foreground a recurring theme: even well-trained agents generate plausible-looking but incorrect arguments, select the wrong tool among semantically similar options, and misinterpret tool outputs ([9]; [10]).

2.2 Benchmarking Agentic Failure

As agents matured, benchmark design shifted from single-call accuracy toward long-horizon, multi-step evaluation. AgentBench introduced eight interactive environments spanning operating systems, databases, and knowledge graphs, providing one of the first systematic comparisons of proprietary and open-source models acting as agents [7]. WebArena went further by embedding agents in fully functional, self-hosted websites rather than static or synthetic pages, evaluating task completion by functional correctness of the resulting environment state rather than by matching a reference action sequence [23]. The low absolute success rates reported on these benchmarks—often below 15–40% for frontier models on realistic long-horizon tasks—indicate that failure, not success, is the empirically dominant outcome for unassisted long-horizon agent execution ([7]; [23]).

Table **??** summarizes representative research approaches, evaluation settings, and major findings relevant to agentic failure and reliability.

More recent work has moved from aggregate success rates toward structured failure analysis. Cemri et al. [2] built the Multi-Agent System Failure Taxonomy (MAST) from 150 hand-annotated traces (validated by an inter-annotator agreement of $\kappa = 0.88$) and scaled it to more than 1,600 traces using an LLM-as-judge pipeline, finding that specification and system-design issues account for roughly 42% of observed failures, inter-agent misalignment for roughly 37%, and inadequate verification for roughly 21% [2]. Independently, work on single-agent trajectories introduced the AgentErrorTaxonomy, organizing failure modes into planning, tool-use, memory, and reflection modules and showing that error propagation—an early mistake cascading into later, compounding errors—is the primary bottleneck to agent reliability in long-horizon tasks (uncredited authorship in source metadata; arXiv:2509.25370). Agent-SafetyBench separately catalogued ten typical unsafe-behavior failure

modes across LLM agents and found that models frequently fabricate parameters when information is insufficient and often ignore the risks of invoking harmful tools ([1], arXiv:2412.14470).

2.3 Security and Adversarial Failure

A distinct branch of the literature treats tool-calling agents as an expanded attack surface. Because agents combine privileged tool access, exposure to untrusted external content (web pages, documents, API responses), and the ability to take consequential external actions, security researchers describe this combination as a "lethal trifecta" that enables complete workflow compromise when any one channel is manipulated (survey discussion in [3], and related MDPI review). InjectAgent formalized this risk by benchmarking indirect prompt injection attacks embedded in tool outputs across a large set of simulated tool-integrated agent scenarios, showing that agents frequently execute attacker-injected instructions returned inside otherwise legitimate tool results [22]. ToolEmu proposed a language-model-emulated sandbox for systematically identifying the risks of LM agents interacting with realistic but simulated tools without requiring costly real-world deployment [13]. Wallace et al. [19] proposed an instruction hierarchy intended to make models more robust to injected instructions embedded in tool outputs by explicitly ranking the trust level of different input sources. These security findings matter directly for the checkpoint-free setting: a human reviewing each tool call and its output has some chance of noticing an anomalous instruction; an agent proceeding through a long chain of calls without review does not.

2.4 Human Oversight and Autonomy Frameworks

A separate but converging literature addresses how much autonomy should be granted to an agent and under what governance structure. The Knight First Amendment Institute's autonomy framework defines five levels of agent autonomy characterized by the role a human plays—operator, collaborator, consultant, approver, and observer—and proposes autonomy certificates as a governance mechanism specifying the maximum level at which an agent may operate (Shavit et al.-style framework; [5]). Complementary risk-based frameworks distinguish between human-in-command (HIC), human-in-the-loop (HITL), and human-on-the-loop (HOTL) oversight regimes, arguing that the appropriate regime should be mapped to the risk category of the task rather than applied uniformly (arXiv:2510.09090). Analyses of the NIST AI Risk Management Framework applied to agentic systems argue that existing governance frameworks were designed around systems with human oversight at most interaction points and exhibit structural gaps when applied to agents capable of spawning sub-agents, acquiring new tool capabilities, and executing long-horizon plans with minimal human interaction (NIST AI RMF Agentic Profile analysis, 2026). These frameworks provide the conceptual vocabulary this paper uses to characterize what is lost when checkpoints are removed: the transition from HITL or HOTL oversight to a fully autonomous operator role.

Table 2: Future Research Directions for Robust LLM Agent Systems

Research Direction	**Core Focus**	**Target Failure Class**	**Key Failure Mode / Context**	**Proposed Strategy**
Dynamic Formal Verification	Runtime invariance	Verification	Silent acceptance of invalid environment outputs	Spec-driven runtime checking & rollback mechanisms
Cross-Agent Protocol Contracts	Coordination interfaces	Coordination	Cascading errors from unvalidated inter-agent communications	Typed message contracts with schema verification
Compositional Task Decomposition	Planning verification	Specification	Early goal misinterpretation leading to persistent failure	Hierarchical validation with intermediate checks
Adversarial Stress-Testing	Robustness	Execution	Unintended tool execution under adversarial prompts	Automated execution sandbox dynamic red-teaming
Continuous Behavioral Audit	Post-execution analysis	All Classes	Undetected drift in dynamic operating conditions	Online monitoring with counterfactual tracking

3 Failure Modes in Chained Tool-Calling Agents: A Structured Account

Building on the taxonomies reviewed above, this section organizes failure modes into four interacting categories and explains, for each, how the absence of a human checkpoint changes the failure's severity, detectability, or reversibility. The major failure categories and their corresponding mitigation strategies are summarized in Table 2.

As the number of dependent tool calls increases, the probability of completing the entire chain successfully depends on the reliability of each intermediate step [**cemri2025mast**, 23].

$$P(\text{successful chain}) = \prod_{i=1}^{n} P(S_i \mid S_1, \ldots, S_{i-1}) \tag{1}$$

3.1 Reasoning and Planning Failures

The MAST taxonomy attributes the largest share of observed multi-agent failures—roughly 42%—to specification and system-design issues, including task misinterpretation, ambiguous role definitions, poor task decomposition, and missing termination conditions [2]. In single-agent long-horizon settings, related work diagnosing where agentic systems break similarly finds that planning-related errors, particularly subplanning failures where an agent grasps the primary intent of an instruction but fails to internalize a constraint, modifier, or conditional clause, are a dominant bottleneck, and that these errors are strongly trajectory-dependent: an early subplanning deviation propagates through every subsequent action (arXiv:2604.11978). Without a checkpoint, a plan built on a subtly

Figure 1: Error Propagation in Autonomous Tool-Calling Agents

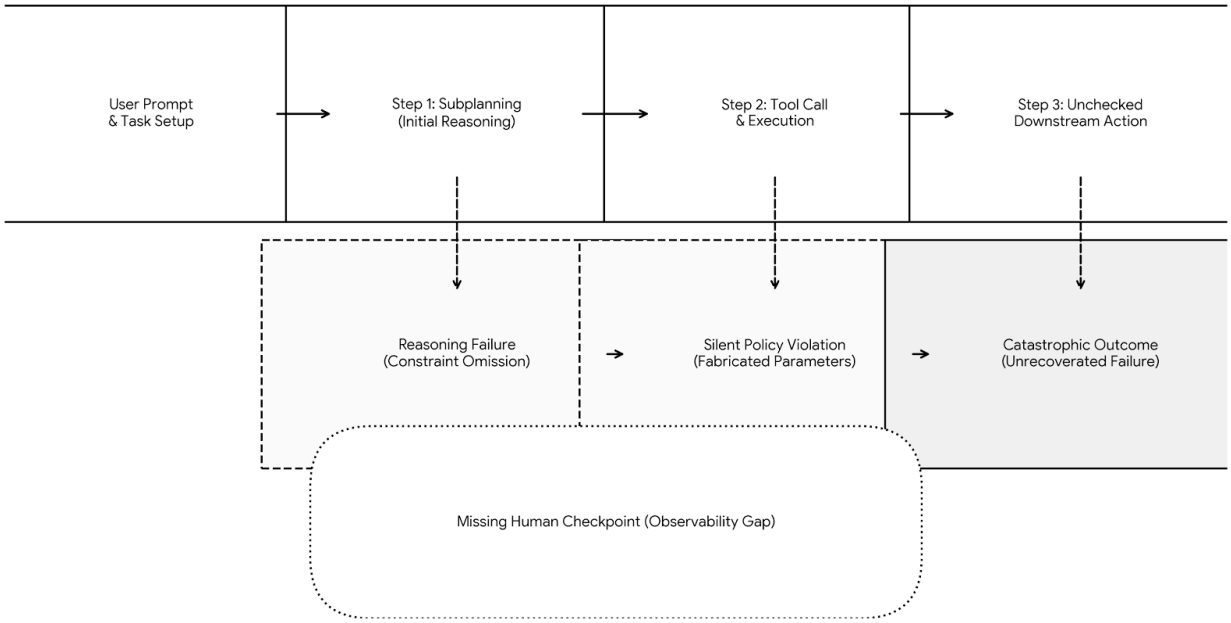


Figure 1: Error propagation cascade in autonomous tool-calling agents. An initial subplanning error propagates uncorrected across intermediate tool calls due to the observability gap, leading to silent policy violations and unrecoverable task failure.

wrong premise is not caught at step one; it is executed to completion, and the agent's own summary of what it did may not reveal the deviation, since the agent typically reports what it believes it accomplished rather than an independently verified account of environment state.

3.2 Tool-Use and Parameter Failures

A second, closely related category concerns errors in the mechanics of tool invocation itself: selecting the wrong tool among semantically similar options, fabricating parameters when required information is missing, and misinterpreting tool outputs. Agent-SafetyBench's failure-mode analysis found that models frequently fabricate parameters to call tools when insufficient information is provided and tend to ignore the potential dangers of invoking harmful tools, identifying these as the two lowest-scoring failure modes among ten evaluated [1]. A related and particularly consequential subtype is the "silent wrong-state" failure, documented in a study of the τ^2-bench airline domain, in which a tool executes a well-formed call that nonetheless violates domain policy—cancelling a booking or changing a passenger count without proper verification—while neither the tool's return value nor the agent's own self-report exposes that a violation occurred; in that study, 78% of observed failures on a budget agent configuration were of this silent, no-error type (arXiv:2607.07405). This subtype is the clearest illustration of why checkpoint-free operation is qualitatively different from checkpointed operation: the very mechanism that would normally catch the error—a visible error message or a human glancing at the proposed action—is absent by construction.

3.3 Memory, Context, and Error Propagation

Long-horizon agents must maintain and update an internal representation of task state across many tool calls, and this representation is a recurring source of failure. Analysis of agent trajectories across ALFWorld, GAIA, and WebShop found that memory and reflection errors are among the most common sources of error propagation, and that a single root-cause failure can cascade into a compounding sequence of downstream errors that ultimately derails the entire trajectory (arXiv:2509.25370). Similarly, work cataloguing structured exception handling in agentic workflows describes error propagation as arising when an upstream mistake is not detected or corrected and is instead carried forward, noting that agents in environments such as OSWorld frequently repeat faulty actions after an early-stage failure without updating their internal state, producing persistent task collapse rather than self-correction (arXiv:2508.07935). Because these cascades unfold silently across many intermediate steps, a human checkpoint placed only at the beginning or end of a task—rather than at intermediate decision points—offers little protection against this failure mode; the deviation is often only visible in the fine-grained trajectory, not in the final output.

3.4 Multi-Agent Coordination Failures

When chained tool-calling is distributed across multiple cooperating agents, additional failure modes emerge from inter-agent communication rather than from any single agent's reasoning. The MAST taxonomy attributes roughly 37% of observed failures to inter-agent misalignment—agents operating on inconsistent assumptions, ignoring peer input, or failing to reconcile conflicting sub-results—and roughly 21% to inadequate verification, where no agent (and no human) checks the correctness of a sub-result before it is used downstream [2]. Related failure-attribution work on multi-agent systems similarly frames the core diagnostic challenge as identifying which single step in a long, distributed trajectory was the causal root of an eventual failure, noting that these systems are "highly sensitive to single-step execution errors that can propagate through agent interactions and lead to cascading failures" (arXiv:2606.03467). In checkpoint-free multi-agent deployments, this diagnostic difficulty is compounded operationally: not only must the failure be attributed after the fact, but no human was positioned to intervene at the point where intervention would have been cheapest.

3.5 Security-Amplified Failures: Prompt Injection Across the Tool Boundary

The failure categories above concern agents acting in good faith on imperfect information. A distinct class of failure arises when the agent's tool-use loop is deliberately exploited. Because a tool-calling agent must treat the content returned by external tools (web pages, documents, emails, other APIs) as part of its reasoning context, an attacker who can influence that returned content can embed instructions the agent will follow—an indirect prompt injection [22]. InjectAgent's benchmark of tool-integrated agents found that models frequently comply with injected instructions returned inside otherwise legitimate tool outputs, and subsequent security surveys describe this as converting prompt injection "from an information threat into an operational threat with physical consequences" once the agent has genuine external tool privileges (survey synthesis referencing [3] and related work). A checkpoint-free chain of tool calls is precisely the setting in which this risk is most acute: each additional unreviewed tool call is an additional opportunity for an attacker-controlled tool

Table 3: Summary of Current Mitigations and Operational Trade-offs

Approach	Primary Defense Mechanism	Key Operational Limitation
Verification Gates	Deterministic pre-execution policy checks	Requires complete prior specification
Verbal Self-Reflection	Prompt-level trajectory feedback	Shared blind spots with base model
Protocol Discipline	Behavioral contracts & orchestration	Architectural refactoring overhead
Tool-Boundary Defenses	Input sanitization & output firewalls	Vulnerable to adaptive prompt injection
Tiered Oversight	Risk-calibrated human intervention	Reviewer fatigue on high-volume tasks

response to redirect the agent's subsequent actions before any human observes the trajectory.

4 Current Approaches and Developments

4.1 Deterministic and Rule-Based Verification Gates

One directly targeted mitigation for silent wrong-state failures is the insertion of deterministic, read-only pre-execution gates that inspect a proposed tool call and the current environment state before permitting a state-changing (write) action to proceed. In the τ^2-bench airline-domain study described above, a four-gate suite raised full-benchmark task success from 29.6% to 42.0% on a smaller model configuration, with the improvement concentrated almost entirely on the subset of tasks where a gate actually fired (a +19.2 percentage-point improvement on firing tasks versus no significant movement on non-firing tasks), and negative-control comparisons showed that such gates add little value when the underlying tools already enforce their own preconditions (arXiv:2607.07405). This line of work suggests that lightweight, domain-specific policy checks inserted directly into the tool-execution path can substitute, in narrow but measurable ways, for a human checkpoint—at the cost of requiring the policy itself to be correctly specified in advance.

4.2 Self-Reflection and Verbal Reinforcement

A second class of mitigation asks the agent to critique its own trajectory. Reflexion equips an agent with a verbal self-reflection mechanism: after a failed or suboptimal attempt, the agent generates a natural-language reflection on what went wrong and carries that reflection forward as additional context on subsequent attempts, functioning as a lightweight, prompt-level analogue to reinforcement learning without updating model weights [16]. Related failure-recovery work builds an explicit taxonomy of error types precisely so that root-cause errors—rather than every surface-level mistake—can be targeted for feedback, since ablation studies show that focusing correction effort on root causes, rather than symptomatic errors, is what drives efficient debugging and meaningful performance gains (arXiv:2509.25370). These approaches reduce, but do not eliminate, the reliance on external checkpoints, since the correction signal still originates from the same model

that produced the error and can therefore share its blind spots.

4.3 Structured Multi-Agent Protocols and Design Discipline

Because MAST attributes the largest share of multi-agent failures to specification and design issues rather than to model capability limits, several recent efforts apply classical software-engineering discipline to agent orchestration. The Software Engineering Multi-Agent Protocol (SEMAP) instantiates explicit behavioral-contract modeling, structured inter-agent messaging, and lifecycle-guided execution with verification atop Google's Agent-to-Agent (A2A) infrastructure, and empirical evaluation using the MAST framework reports up to a 69.6% reduction in total failures for function-level software-development tasks and 56.7% for deployment-level tasks (arXiv:2510.12120). This supports the broader claim, echoed across the multi-agent failure literature, that many observed failures stem from poor system design rather than model performance, and that improving robustness requires better orchestration strategies rather than solely larger or more capable models ([2]; synthesis in secondary literature).

4.4 Security Defenses Against Tool-Boundary Attacks

On the security side, defenses against indirect prompt injection in tool-calling agents fall broadly into input-sanitization approaches, instruction-hierarchy training that teaches the model to weight system and developer instructions above content encountered through tools, and post-hoc firewall or output-filtering layers placed between tool results and the agent's context ([19]; firewall approaches reviewed in arXiv:2510.05244). However, evaluation of these defenses has generally found them to be incomplete: a benchmark of adaptive attacks against eight existing defense mechanisms found that all could be bypassed, with attack success rates exceeding 50% in each case once the adversary adapted to the specific defense (synthesis reported in arXiv:2606.05233 and related sources), and a review of firewall-based defenses explicitly cautions that developers risk over-relying on such defenses without understanding the conditions under which they remain effective (arXiv:2510.05244). Model providers have also begun reporting empirical reductions in browser-agent prompt-injection attack success rates across successive model releases, suggesting incremental but partial progress at the model-training level rather than a solved problem (industry system-card disclosures referenced in arXiv:2606.05233).

4.5 Tiered Human Oversight Models

Rather than treating oversight as binary (checkpoint present or absent), recent governance-oriented work proposes tiered models that scale the intensity of human involvement to the risk of the action. The risk-based alignment framework distinguishes human-in-command, human-in-the-loop, and human-on-the-loop regimes and maps each to a risk category, recommending, for example, human-in-the-loop approval before execution for medium-to-high-risk actions and continuous human-on-the-loop monitoring with anomaly-triggered intervention for lower-risk but still consequential autonomous operation (arXiv:2510.09090). The Knight First Amendment Institute's autonomy-levels framework operationalizes a related idea through five roles a human can occupy relative to an

agent—operator, collaborator, consultant, approver, and observer—and proposes autonomy certificates that specify, and can be audited against, the maximum autonomy level at which a given agent is permitted to operate [5]. These frameworks represent an attempt to reintroduce meaningful human oversight into agent deployment without reverting to a fully checkpointed, single-step-at-a-time interaction model that would negate much of the efficiency benefit of autonomous chaining.
Risk-calibrated oversight can be represented by an intervention threshold, where actions whose estimated risk exceeds the threshold are routed for human review [**kwa2025metr**].

$$I(a) = \begin{cases} 1, & R(a) \geq \tau, \\ 0, & R(a) < \tau, \end{cases} \tag{2}$$

5 Research Challenges and Limitations

Several limitations recur across the literature reviewed in this paper and constrain how confidently its findings generalize.

- Benchmark-to-deployment gap: Most quantitative failure-rate evidence comes from constructed benchmarks (WebArena, AgentBench, τ^2-bench, MAST-Data) rather than from audited real-world production deployments, and benchmark task distributions may not reflect the tail risks that matter most in practice ([7]; [23]; [2]).
- Attribution difficulty: Because errors propagate and compound across long trajectories, identifying the true root-cause step of a failure is itself an unsolved research problem; several papers reviewed here (arXiv:2509.25370; arXiv:2606.03467; arXiv:2605.08715) are themselves attempts to build attribution methodology, indicating the field has not yet converged on a standard approach.
- LLM-as-judge annotation reliability: Several large-scale failure datasets, including MAST-Data, rely on LLM-as-judge pipelines to scale annotation beyond what human review can cover; while these pipelines report high agreement with human annotators in the originating studies [2], this evaluation methodology has an inherent circularity risk when the judge and the agent share failure-prone characteristics.
- Rapidly shifting capability baselines: Because time-horizon and reliability metrics are improving quickly and non-uniformly across task types—Kwa et al. [6] report a roughly seven-month doubling time for 50%-reliability task horizon—failure-rate findings tied to specific model generations risk becoming stale, complicating durable policy or engineering recommendations.
- Security-mitigation arms race: Evaluations showing that adaptive attackers can bypass essentially all tested indirect-prompt-injection defenses (arXiv:2606.05233) indicate that current security mitigations should be treated as risk-reducing rather than risk-eliminating, which has implications for how much autonomy can responsibly be granted even when a defense is nominally in place.
- Fragmented taxonomies: Multiple independent taxonomies (MAST, AgentErrorTaxonomy, Agent-SafetyBench's ten failure modes, and domain-specific taxonomies such as SHIELDA)

currently coexist without a unifying standard, making cross-study comparison of failure rates difficult.

6 Research Gaps and Future Directions

6.1 Standardized, Cross-Domain Failure Taxonomy and Reporting

Given the proliferation of partially overlapping taxonomies (Section 5), a clear gap exists for a validated, cross-domain failure taxonomy with agreed-upon severity and detectability dimensions, analogous to how vulnerability severity is standardized in software security (e.g., CVSS). Such a taxonomy would allow failure rates reported for one agent framework or benchmark to be meaningfully compared with another, and would support longitudinal tracking of whether specific failure categories are actually declining as models and tooling improve.

6.2 Intermediate-Trajectory Oversight Mechanisms

Because several of the most consequential failure modes reviewed here—silent wrong-state failures, cascading error propagation, and indirect prompt injection—are specifically invisible to checkpoints placed only at the start or end of a task, an important research direction is the design of lightweight, low-latency oversight mechanisms that operate at intermediate points in a trajectory without eliminating the efficiency benefits of autonomous chaining. The deterministic pre-execution gates studied in the τ^2-bench work (arXiv:2607.07405) are an early example of this class of solution, but their generalization beyond narrow, well-specified domains (e.g., open-ended research or creative tasks where "correct state" is not crisply definable) remains largely unstudied.

6.3 Risk-Calibrated Autonomy Assignment

Existing autonomy-level and human-oversight frameworks ([5]; arXiv:2510.09090) provide conceptual scaffolding but have not yet been empirically validated against measured failure rates across task domains. A concrete research gap is the construction of empirically grounded mappings from task risk and reversibility characteristics to a recommended autonomy tier and checkpoint frequency, ideally validated against observed incident rates rather than derived purely from first-principles risk reasoning.

6.4 Joint Modeling of Capability Growth and Oversight Requirements

Because task-completion time horizons are increasing rapidly [6], the number of chained tool calls that can occur between points of human awareness is also increasing, even if per-step error rates fall. Future work should explicitly model the relationship between agent capability growth, checkpoint frequency, and expected undetected-failure rate, rather than treating capability improvement and

oversight design as independent research tracks. This is particularly important because Kwa et al. note that increased time horizon is driven substantially by improved error-recovery and reliability rather than by raw task difficulty tolerance, which may create a false sense that improving capability alone reduces the need for oversight.

6.5 Robust, Non-Bypassable Security Defenses at the Tool Boundary

Given documented adaptive bypass rates exceeding 50% against current indirect-prompt-injection defenses (synthesis in arXiv:2606.05233), a clear technical gap remains for defenses with formal or near-formal guarantees at the tool boundary, potentially drawing on the instruction-hierarchy approach [19] combined with architectural isolation techniques such as sandboxing high-privilege tool calls behind mandatory verification, rather than relying primarily on model-level training to resist injected instructions.

6.6 Human Factors in Checkpoint Design

Finally, most of the literature reviewed treats "human checkpoint" as a binary design choice rather than studying how checkpoint placement, framing, and cognitive load affect whether a human reviewer actually catches an error when one is presented. Given evidence that autonomy framing itself changes user trust and reliance (arXiv:2510.04465), an underexplored direction is empirical human-factors research on what checkpoint designs actually improve error-catch rates in practice, as opposed to checkpoints that exist nominally but are rubber-stamped due to reviewer fatigue or opaque agent trajectories.

7 Conclusion

Chained tool-calling agents extend the demonstrated reasoning and tool-use capabilities of large language models into longer, more autonomous trajectories, and the empirical trend toward longer unsupervised task-completion horizons [6] shows no sign of reversing. This paper has synthesized recent literature to argue that removing human checkpoints from these trajectories does not merely remove a convenience step; it removes the primary mechanism by which several of the most consequential failure modes—silent wrong-state actions, cascading error propagation, inter-agent misalignment, and adversarial prompt injection across the tool boundary—are currently caught in practice. Taxonomic work such as MAST [2], Agent-SafetyBench [1], and the AgentErrorTaxonomy (arXiv:2509.25370) converge on the finding that a substantial share of failures originate in specification and system-design weaknesses rather than in raw model capability limits, which is a cautiously optimistic finding: it suggests that better orchestration, verification gates, and tiered oversight models can meaningfully reduce failure rates without necessarily waiting on further model scaling. At the same time, evidence that adaptive attackers can bypass most current security defenses (arXiv:2606.05233) and that silent policy-violating actions can occur without any visible error signal (arXiv:2607.07405) indicates that current mitigations are risk-reducing rather than risk-eliminating. The central implication for both researchers and practitioners is that autonomy and oversight should

be co-designed: as chained tool-calling agents are granted longer and less-supervised trajectories, the mechanisms for intermediate verification, failure attribution, and adversarial robustness need to scale correspondingly, rather than being retrofitted after incidents occur.

Complete References

[1] Agent-SafetyBench authors [Zhang, Z. et al.] (2024). Agent-SafetyBench: Evaluating the Safety of LLM Agents. arXiv preprint. arXiv:2412.14470. https://arxiv.org/abs/2412.14470

[2] Cemri, M., Pan, M. Z., Yang, S., et al. (2025). Why Do Multi-Agent LLM Systems Fail? Presented at NeurIPS 2025. arXiv preprint arXiv:2503.13657. https://arxiv.org/abs/2503.13657

[3] Ferrag, M. A., et al. (2025). From Prompt Injections to Protocol Exploits: Threats in LLM-Powered AI Agents Workflows. ScienceDirect (Computers & Security / related venue). https://www.sciencedirect.com/science/article/pii/S2405959525001997

[4] Kasirzadeh, A., & Gabriel, I. (2025). AI Agent Autonomy Framework [as operationalized in Autonomy Reshapes How Personalization Affects Privacy Concerns and Trust in LLM Agents]. arXiv preprint arXiv:2510.04465. https://arxiv.org/abs/2510.04465

[5] Knight First Amendment Institute (2025). Levels of Autonomy for AI Agents. Knight First Amendment Institute at Columbia University. https://knightcolumbia.org/content/levels-of-autonomy-for-ai-agents-1

[6] Kwa, T., West, B., Becker, J., Deng, A., Garcia, K., Hasin, M., Jawhar, S., Kinniment, M., Rush, N., Von Arx, S., Bloom, R., Broadley, T., Du, H., Goodrich, B., Jurkovic, N., Miles, L. H., Nix, S., Lin, T., Painter, C., Parikh, N., Rein, D., Sato, L. J. K., Wijk, H., Ziegler, D. M., Barnes, E., & Chan, L. (2025). Measuring AI Ability to Complete Long Software Tasks. Model Evaluation & Threat Research (METR). arXiv preprint arXiv:2503.14499. https://arxiv.org/abs/2503.14499

[7] Liu, X., Yu, H., Zhang, H., Xu, Y., Lei, X., Lai, H., Gu, Y., Ding, H., Men, K., Yang, K., Zhang, S., Deng, X., Zeng, A., Du, Z., Zhang, C., Shen, S., Zhang, T., Su, Y., Sun, H., Huang, M., Dong, Y., & Tang, J. (2023). AgentBench: Evaluating LLMs as Agents. Presented at ICLR 2024. arXiv preprint arXiv:2308.03688. https://arxiv.org/abs/2308.03688

[8] NIST AI Risk Management Framework: Agentic Profile analysis (2026). Cloud Security Alliance Lab Space. https://labs.cloudsecurityalliance.org/agentic/agentic-nist-ai-rmf-profile-v1/

[9] Patil, S. G., Zhang, T., Wang, X., & Gonzalez, J. E. (2023). Gorilla: Large Language Model Connected with Massive APIs. arXiv preprint arXiv:2305.15334. Also published in Advances in Neural Information Processing Systems (NeurIPS 2024), 37, 126544–126565. https://arxiv.org/abs/2305.15334

[10] Qin, Y., Liang, S., Ye, Y., Zhu, K., Yan, L., Lu, Y., Lin, Y., Cong, X., Tang, X., Qian, B., Zhao, S., Hong, L., Tian, R., Xie, R., Zhou, J., Gerstein, M., Li, D., Liu, Z., & Sun, M. (2023). ToolLLM: Facilitating Large Language Models to Master 16000+ Real-World APIs. arXiv preprint arXiv:2307.16789. https://arxiv.org/abs/2307.16789

[11] Reason Less, Verify More: Deterministic Gates Recover a Silent Policy-Violation Failure Mode in Tool-Using LLM Agents. Reddy, V., Challaram, S. R., & Basu, A. arXiv preprint arXiv:2607.07405. https://arxiv.org/pdf/2607.07405

[12] Risk-Based Framework for AI and Human Oversight (2025). AI and Human Oversight: A Risk-Based Framework for Alignment. arXiv preprint arXiv:2510.09090. https://arxiv.org/abs/2510.09090

[13] Ruan, Y., Dong, H., Wang, A., Pitis, S., Zhou, Y., Ba, J., Dubois, Y., Maddison, C. J., & Hashimoto, T. (2024). Identifying the Risks of LM Agents with an LM-Emulated Sandbox (ToolEmu). Presented at ICLR 2024. arXiv preprint arXiv:2309.15817. https://arxiv.org/abs/2309.15817

[14] Schick, T., Dwivedi-Yu, J., Dessì, R., Raileanu, R., Lomeli, M., Hambro, E., Zettlemoyer, L., Cancedda, N., & Scialom, T. (2023). Toolformer: Language Models Can Teach Themselves to Use Tools. Advances in Neural Information Processing Systems (NeurIPS 2023). arXiv preprint arXiv:2302.04761. https://arxiv.org/abs/2302.04761

[15] SHIELDA: Structured Handling of Exceptions in LLM-Driven Agentic Workflows. arXiv preprint arXiv:2508.07935. https://arxiv.org/abs/2508.07935

[16] Shinn, N., Cassano, F., Berman, E., Gopinath, A., Narasimhan, K., & Yao, S. (2023). Reflexion: Language Agents with Verbal Reinforcement Learning. Advances in Neural Information Processing Systems (NeurIPS 2023). arXiv preprint arXiv:2303.11366. https://arxiv.org/abs/2303.11366

[17] StepFinder: A Temporal Semantic Framework for Failure Attribution in Multi-Agent Systems. Zhu, T., Wu, Y., Jin, W., Li, Y., & Huang, G. arXiv preprint arXiv:2606.03467. https://arxiv.org/abs/2606.03467

[18] The Long-Horizon Task Mirage? Diagnosing Where and Why Agentic Systems Break (2026). arXiv preprint arXiv:2604.11978. https://arxiv.org/abs/2604.11978

[19] Wallace, E., Xiao, K., Leike, J., Weng, L., Heidecke, J., & Beutel, A. (2024). The Instruction Hierarchy: Training LLMs to Prioritize Privileged Instructions. arXiv preprint arXiv:2404.13208. https://arxiv.org/abs/2404.13208

[20] Where LLM Agents Fail and How They Can Learn From Failures (2025). Introduces AgentErrorTaxonomy and AgentErrorBench. arXiv preprint arXiv:2509.25370. https://arxiv.org/abs/2509.25370

[21] Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., & Cao, Y. (2023). ReAct: Synergizing Reasoning and Acting in Language Models. Presented at the Eleventh International Conference on Learning Representations (ICLR 2023). arXiv preprint arXiv:2210.03629. https://arxiv.org/abs/2210.03629

[22] Zhan, Q., Liang, Z., Ying, Z., & Kang, D. (2024). InjectAgent: Benchmarking Indirect Prompt Injections in Tool-Integrated Large Language Model Agents. arXiv preprint arXiv:2403.02691. https://arxiv.org/abs/2403.02691

[23] Zhou, S., Xu, F. F., Zhu, H., Zhou, X., Lo, R., Sridhar, A., Cheng, X., Ou, T., Bisk, Y., Fried, D., Alon, U., & Neubig, G. (2023). WebArena: A Realistic Web Environment for Building Autonomous Agents. Presented at ICLR 2024. arXiv preprint arXiv:2307.13854. https://arxiv.org/abs/2307.13854

[24] Software Engineering Multi-Agent Protocol (SEMAP): Towards Engineering Multi-Agent LLMs: A Protocol-Driven Approach (2025). arXiv preprint arXiv:2510.12120. https://arxiv.org/abs/2510.12120

[25] Adaptive Evaluation of Out-of-Band Defenses Against Prompt Injection in LLM Agents (2026). arXiv preprint arXiv:2606.26479. https://arxiv.org/abs/2606.26479

[26] Domain-Conditioned Safety in Frontier Computer-Using Agents: A 793-Episode Browser Benchmark (2026). arXiv preprint arXiv:2606.05233. https://arxiv.org/abs/2606.05233

[27] Indirect Prompt Injections: Are Firewalls All You Need, or Stronger Benchmarks? (2025). arXiv preprint arXiv:2510.05244. https://arxiv.org/abs/2510.05244

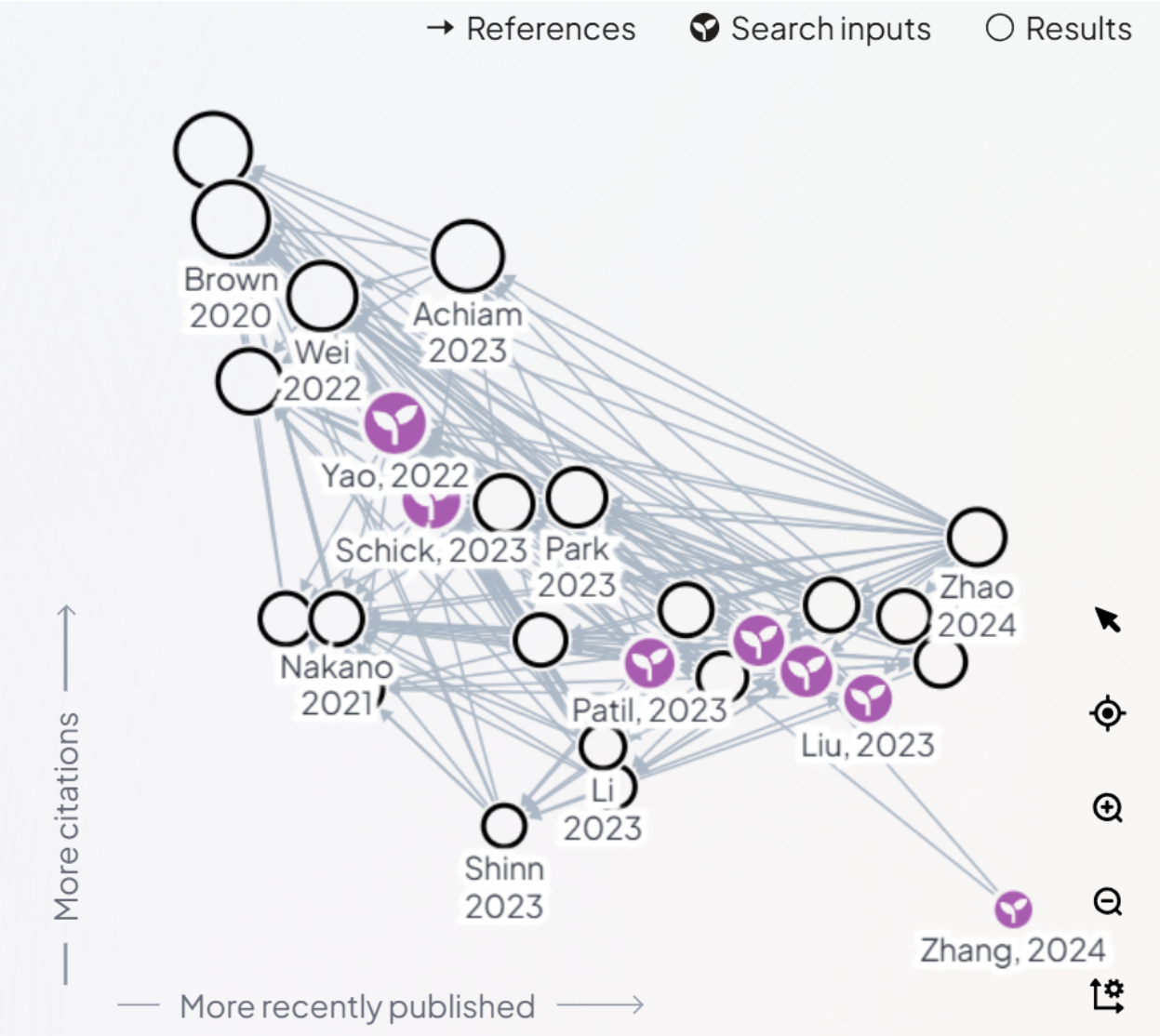


Figure 2: Citation mapping showing relationships across foundational and recent LLM agent literature.